Supplementary Material

When Does Reconfiguration Awareness Pay Off? An Exact-Oracle Study of Learned Link Sleeping in Software-Defined Networks

Adeel Ahmad, Toqeer Ali Syed, Ali Akarma

This file holds the algorithms, the secondary experiments, and the tables and figures that complement the article. Equation, section, and table numbers without the prefix S refer to the article. All values are generated from the experiment files in `https://github.com/adeliusa486/-lira-sdn-energy`.

S1. Related Work

Energy-Aware Networking and Sleep Modes

Surveys of green networking classify techniques into adaptive link rate, sleeping, and energy-aware routing [1, 2]. Chiaraviglio *et al.* formulated ISP energy minimization as an integer program and proposed heuristics that switch off nodes and links subject to capacity and connectivity constraints [3]. GreenTE maximizes the number of links that can sleep under utilization and delay constraints [4], ElasticTree applies the same principle to data-center networks [5], and EATe spreads traffic online so that lightly loaded links can sleep [6]. Power models for such studies rely on measured device profiles [7, 8, 9]. In SDN, energy-aware routing has been formulated as integer programs with heuristics for carrier-grade Ethernet [10], and a survey of SDN-specific methods is given in [11].

Learning for Energy-Efficient Networking

MER-SDN uses machine learning to predict the parameters of an energy-aware routing heuristic [12]. Reinforcement learning has been applied in an SDN controller to trade packet delay against energy [13] and to compute energy-saving routes in data-center networks [14]. In cellular networks, deep reinforcement learning combined with traffic forecasting controls base-station sleep modes [15], a problem that shares the on-off structure of link sleeping. Deep reinforcement learning has also been used to put switches and links of data-center networks to sleep [16] and to route traffic in SDN [17], and energy-aware traffic engineering has been adapted to hybrid SDN deployments [18]. A recent study of two European ISP networks shows that more than a third of the links could sleep without congestion [19], which confirms that the

opportunity measured on research backbones exists in operational networks. The closest works to ours replace the optimizer with a learned mapping from traffic matrices to precomputed configurations and report large speedups [20, 21]. We reimplement this snapshot approach as baseline B4 with our own library, features, and classifier, because we did not have access to the original code. The comparison therefore concerns the snapshot principle and not the exact published systems.

Learning to Optimize and Learned Traffic Engineering

Machine learning for combinatorial optimization treats problem instances as data and learns to imitate expert decisions or to replace parts of a solver [22]. In traffic engineering, learning-accelerated and decomposition-based solvers [23, 24, 25, 26], learned routing [27], deep reinforcement learning with graph neural networks [28, 29], and neural models of network performance [30] have shown that learned components can approach solver quality at much lower latency. Most of these methods do not price the change from one configuration to the next. FlexDATE is an exception: it limits the disturbance of rerouting in reinforcement-learning traffic engineering [31], but it does not put links to sleep. LiRA instead imitates an oracle that is exact for the sequential objective, which the finite library makes tractable offline.

Reconfiguration Cost, Device Lifetime, and Switching Costs

Consistent-update abstractions [32] and update scheduling [33] show that changing forwarding state is neither free nor instantaneous. Bonetto *et al.* included reconfiguration costs in multi-period power-aware logical topology design [34]. Chiaraviglio *et al.* modeled how sleep modes affect device lifetime through temperature and power-state transitions [35, 36], and lifetime-aware provisioning has been studied for optical backbones [37]. Online problems with switching costs are studied as metrical task systems [38] and smoothed online convex optimization [39], with dynamic right-sizing of data centers as a well-known application [40]. Our setting is a metrical task system with a finite state space whose offline optimum is computable, which makes imitation learning natural [41]. Learning-augmented online algorithms combine a learned predictor with a robust online rule, with guarantees for caching [42], general metrical task systems [43, 44], and smoothed online optimization [45]. They show that following a predictor blindly can be costly, which is the flapping we measure, and that a rule which holds its state is the natural robust

fallback. Our hysteresis and static baselines are simple instances of such rules. Concept-drift adaptation [46, 47] with detectors such as ADWIN [48] and cumulative-sum tests [49] provides the tools for deciding when to retrain, and drift is known to degrade learned models in network security [50].

S2. Algorithms

Algorithm S1 summarizes offline training and Algorithm S2 the online loop. In the emulation (Section S12), reconfigurations follow a make-before-break order: links that the new configuration needs are woken first, new rules are installed, and only then are unused links put to sleep. The flow-level simulation charges each change through R and W and does not model the transient.

Algorithm S1 LiRA Offline Training

Input: training traces $D_{1:T}$, validation traces, prices λ_R, λ_W, rounds N
Output: library $\mathcal{C}$, policy π

$\mathcal{C} \leftarrow$ BUILDLIBRARY$(D_{1:T})$ ▷ ILP, NSGA-II, set cover
precompute $E(c, D_t)$, feasibility, and R, W for all pairs
compute $Q_t(c', c)$ and $V_t(c)$ by recursion (8)
$\mathcal{D} \leftarrow \{(s_t, c_t^\star)\}$ on the oracle path (s_t contains $c_{t-1}^\star$), weights w
train π on $\mathcal{D}$
for $n = 1$ to N **do**
    roll out π with the safety layer and expert mixing β_n; record (t, c_{t-1})
    relabel the visited pairs from $Q_t(c_{t-1}, \cdot)$ and add them to $\mathcal{D}$
    retrain π on $\mathcal{D}$
end for
return $\mathcal{C}$ and the policy with the lowest validation cost

S3. Limitations and Threats to Validity

- *Traffic.* The only series with strong flapping is semi-synthetic. We vary its noise structure (Section S15), but no real trace with such fluctuations was available to us. The evaluation on real traffic covers two research and education backbones, Abilene (2004) and GÉANT (2005), and both traces are old. The Nobel-Germany test split covers only 7.5 days, so tests on per-day costs have little power.

Algorithm S2 LiRA Online Control Loop

Input: $\mathcal{C}$, π, threshold θ, initial c_0

for each interval $t = 1, 2, \ldots$ **do**
  build s_t from the traffic estimate, c_{t-1}, and its dwell time
  $c \leftarrow \pi(s_t)$
  if $\mathrm{MLU}(c, \hat{D}_t) > \theta$ **then**
    $c \leftarrow$ fallback rule (9)
  end if
  if $c \neq c_{t-1}$ **then**
    wake links in $x \setminus x_{t-1}$; install new rules
    sleep links in $x_{t-1} \setminus x$
  end if
  $c_t \leftarrow c$; observe D_t; update detectors with ρ_t from (1)
  if a detector fires **then**
    schedule retraining on the last seven days (Section S4)
  end if
end for

- *Power and wear.* The port power values and the wear parameters are assumed and not measured. The conclusion that the optimal configuration is stable depends on idle port power dominating dynamic power. A router with stronger load-proportional power, such as the high-dynamic-power scenario scaled further, may change this.

- *Baselines.* B4 is our reimplementation of the snapshot principle, not the published systems. B2 is restricted to the library. We did not solve the full multi-period mixed-integer program, but the per-interval bound limits what it can gain over the oracle (Section 6.3). B1, B3, and B5 were evaluated on subsamples. We did not train a deep reinforcement learning baseline.

- *Information.* The main results let every online method see the traffic of the interval it configures. With the previous interval's traffic, costs are unchanged but the safety layer admits violations on Nobel-Germany (Section S16). We did not evaluate traffic forecasts or estimation error from link loads.

- *Drift.* The injected shifts did not change the optimal configuration, and the detectors fired independently of them, so the drift study is inconclusive. Link failures and large library refreshes were not evaluated.

- *Emulation.* The emulation is small (17 switches, one day, compressed time), uses a modeled wake-up delay, single-path routing per demand, and no real line cards, and it reports a single run for LiRA.

- *Statistics.* Seeds vary only the learner. The traffic traces are fixed and the per-day tests use few days, so no comparison survives a correction over all settings.

- *Learned component.* In 30 of 60 LiRA runs the trained policy is constant, in 56 of them the selected policy predates DAgger, and the static rule equals LiRA in 11 of 12 settings. A learned policy that does more than hold a configuration is therefore evaluated only in the Nobel-Germany stress scenario (898.6 against 894.3 for the oracle, five seeds). On GÉANT the classifier is not constant as a model, but its output is constant on the test split.

S4. Drift Detection and Retraining

The oracle cannot be computed online, but a myopic regret proxy can. After each decision, the controller computes

$$\begin{aligned}\rho_t = & E(c_t, D_t) - \min_{c \in \mathcal{C}:\, \mathrm{MLU}(c, D_t) \le \theta} E(c, D_t) \\ & + \mathbb{1}[\mathrm{MLU}(c_t, D_t) > \theta],\end{aligned} \tag{1}$$

the energy lost against the best feasible configuration plus a violation indicator. ADWIN [48] monitors ρ_t, and a Page–Hinkley test [49] monitors the deseasonalized log of the total traffic volume, that is, the log volume minus its training mean for the same time of week. When a detector fires, the policy is retrained on the last seven days with oracle labels computed on that window. Because the window just after a change still contains mostly old data, retraining is repeated one and three days later. A library refresh, which adds integer-program solutions for eight clusters of the recent window, is available as an option.

S5. Dependence on the Snapshot Classifier

The amount of flapping depends strongly on the classifier. On Nobel-Germany in the base scenario, B4 with XGBoost changes configuration 12.8

times per day, LightGBM 10.1, a random forest 0.5, and a multilayer perceptron 44.8 (Table S1). The random forest is nearly as stable as LiRA. Averaging over trees is a plausible reason, which we did not test. The perceptron flaps so much that its cost (1093.7) is far above that of LiRA (875.9). LiRA reaches the same cost with every backend, because its policy is constant in this scenario. A sequence-aware method is therefore most useful when the snapshot classifier is the kind that flaps, and it is least useful when a smoother classifier or a hysteresis guard is available.

Table S1: Choice of Model on Nobel-Germany (Base Scenario, Seed 0). Chg/d: Changes per Day

	B4 (snapshot)			LiRA	
Model	Chg/d	J	Latency (ms)	Chg/d	J
XGBoost	12.80	937.2	1.09	0.27	875.9
LightGBM	10.13	924.1	1.61	0.27	875.9
Random forest	0.53	877.2	17.22	0.27	875.9
MLP (2×256)	44.80	1093.7	0.72	0.27	875.9

S6. Ablation, Library Size, and Threshold

Table S2 removes one component of LiRA at a time on Nobel-Germany at the default price. It uses seeds 0 to 2, so the full policy's stress cost differs from the five-seed value in Table 2. In the high-load scenario the expert is nearly constant, so all variants that keep the safety layer reach the oracle cost. In the stress scenario the full policy costs 897.4 against 894.3 for the oracle. Removing the previous-configuration input raises the cost to 902.0, removing the cost-sensitive weights to 898.6, and removing DAgger to 899.0. Removing the lag features lowers it to 895.6. The differences are small and of mixed sign, and we did not test them for significance. Removing the safety layer lowers the cost to 889.9 because the policy then ignores the bound, and it causes 0.17% of the intervals to violate θ. Because J carries no penalty for violations, this cost is not comparable, and the layer is needed. On Abilene and GÉANT all variants coincide with the oracle because the expert is constant.

Table S3 shows the effect of the library size on Nobel-Germany. Requested sizes above 51 yield the same library, because the candidate pool contains only 51 configurations. Every library size covers all test matrices. A larger

library lowers the cost of B4 (from 939.3 at $K = 8$ to 935.7 at the largest size), but hysteresis and LiRA stay within 0.2% of the oracle cost at every size. Table S4 varies the utilization threshold. At $\theta = 0.6$ B4 changes configuration 10.4 times per day and LiRA 0.40 times. For $\theta \geq 0.7$, each method gives the same result at every threshold on Nobel-Germany, because the same configurations are optimal and feasible. Table S4 uses seed 0, so its B4 change rate differs from the five-seed mean of Table 1. On Abilene and GÉANT, the threshold has no effect between 0.6 and 0.9. LiRA never violates the bound when the library covers the traffic.

Table S2: Ablation of LiRA on Nobel-Germany at the Default Price (Mean over Seeds 0 to 2)

Variant	S (%)	Changes/d	MLU viol. (%)	J
Nobel-Germany, high load				
Oracle O*	27.57	0.80	0.00	880.9
LiRA (full)	27.57	0.80	0.00	880.9
no lag features	27.57	0.80	0.00	880.9
no previous config.	27.57	0.80	0.00	880.9
no cost-sensitive weights	27.57	0.80	0.00	880.9
no DAgger	27.57	0.80	0.00	880.9
no safety layer	27.58	0.00	0.14	877.3
Nobel-Germany, stress				
Oracle O*	27.35	0.80	0.00	894.3
LiRA (full)	27.37±0.03	0.93±0.33	0.00	897.4±2.2
no lag features	27.36±0.03	0.84±0.19	0.00	895.6±5.0
no previous config.	27.40	1.07	0.00	902.0
no cost-sensitive weights	27.37±0.03	1.07	0.00	898.6±7.8
no DAgger	27.37±0.03	0.89±0.38	0.00	899.0±9.1
no safety layer	27.40±0.07	0.00	0.17±0.07	889.9±1.6

Table S3: Library Size on Nobel-Germany: Test Coverage and Total Cost J of Each Method (Base Scenario, Seed 0)

K	Test coverage (%)	B4	B6	LiRA	O*
8	100.0	939.3	875.3	875.9	874.7
16	100.0	938.6	875.3	875.9	874.7
32	100.0	937.2	875.3	875.9	874.7
64 (51)	100.0	935.7	875.3	875.8	874.7
128 (51)	100.0	935.7	875.3	875.8	874.7

Table S4: Utilization Threshold θ (Base Scenario, Seed 0)

Topology	θ	All-on viol. (%)	$S_{O^\star}$ (%)	B4 chg/d	LiRA chg/d	LiRA viol. (%)	LiRA J
Abilene	0.6	0.00	23.98	0.00	0.00	0.00	7226.7
Abilene	0.7	0.00	23.98	0.00	0.00	0.00	7226.7
Abilene	0.8	0.00	23.98	0.00	0.00	0.00	7226.7
Abilene	0.9	0.00	23.98	0.00	0.00	0.00	7226.7
GÉANT	0.6	0.00	29.85	0.00	0.00	0.00	603.7
GÉANT	0.7	0.00	29.85	0.00	0.00	0.00	603.7
GÉANT	0.8	0.00	29.85	0.00	0.00	0.00	603.7
GÉANT	0.9	0.00	29.85	0.00	0.00	0.00	603.7
Nobel-Germany	0.6	0.09	27.63	10.40	0.40	0.00	876.0
Nobel-Germany	0.7	0.05	27.63	12.80	0.27	0.00	875.9
Nobel-Germany	0.8	0.05	27.63	12.80	0.27	0.00	875.9
Nobel-Germany	0.9	0.05	27.63	12.80	0.27	0.00	875.9

S7. Wear and Lifetime

Fig. S1 shows the mean acceleration factor of the decision sequences on Nobel-Germany for $\alpha = 0.5$ and varying cycling weight χ (the sequences are fixed, and only the model changes). Every method stays below the always-on value of one for $\alpha = 0.5$ and all χ up to 0.1, but B4 degrades fastest: at $\chi = 0.1$ its mean factor is 0.94, against 0.85 for LiRA. At the default parameters (seed 0) the most-cycled link of B4 has a factor of 1.117, whereas the most-cycled link of LiRA has 1.0024 and those of B6, B7, and the oracle 1.000. When sleeping brings no benefit ($\alpha = 1$), every transition lowers lifetime, and every method that switches exceeds the always-on factor, B4 by the largest margin. Absolute lifetime figures depend on assumed parameters and should be read as relative comparisons.

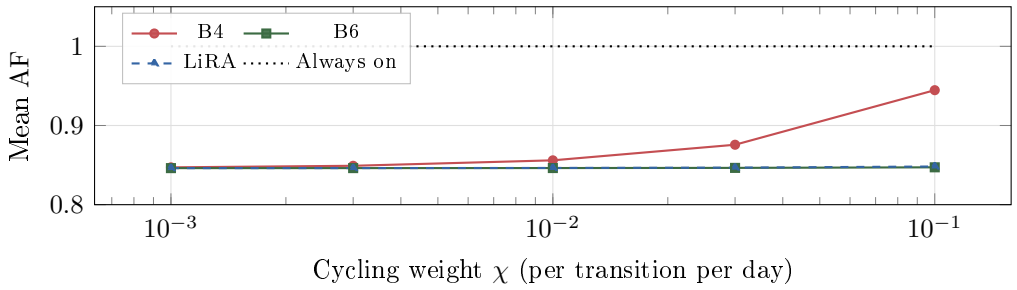


Figure S1: Mean acceleration factor of the decision sequences on Nobel-Germany for $\alpha = 0.5$. The factor of the always-on network is one.

S8. Decision Time and Scalability

The library-based decisions are fast. The median decision time of B4 is 0.99 ms on Abilene, 0.95 ms on Nobel-Germany, and 1.60 ms on GÉANT. LiRA needs less than 0.01 ms where its policy is constant and 1.571 ms where a tree model is evaluated. The safety check takes at most 0.36 ms even on a 65-node topology. Solving the integer program per traffic matrix (B1) takes a median of 537 ms on Nobel-Germany and 3.5 s on GÉANT, NSGA-II (B3) takes 28 s and 42 s, and the greedy heuristic is faster but gives away part of the savings (Table S5). On the subsample, the per-snapshot integer program reaches the same savings as the library snapshot optimum to within 0.007 percentage points. On larger SNDlib topologies the integer program takes 1.6 s for Germany50 and 11.4 s for TA2 (Table S6). These times fit five-minute intervals on the topologies we study, so speed alone does not require a learned controller here. The dynamic-programming oracle needs 3.6 s for all 48 108 Abilene intervals with $K = 32$, so prices can be explored offline.

Table S5: Per-Snapshot Optimization Baselines on a Test Subsample

Topology	Method	n	S (%)	Median time (s)
Abilene	Library snapshot optimum	60	23.98	–
Abilene	B1 per-snapshot ILP	60	23.98	0.53
Abilene	B3 NSGA-II	15	23.98	253.97
Abilene	B5 greedy	60	23.55	0.04
GÉANT	Library snapshot optimum	60	29.85	–
GÉANT	B1 per-snapshot ILP	60	29.85	3.53
GÉANT	B3 NSGA-II	60	29.56	41.56
GÉANT	B5 greedy	60	27.16	0.24
Nobel-Germany	Library snapshot optimum	60	27.63	–
Nobel-Germany	B1 per-snapshot ILP	60	27.63	0.54
Nobel-Germany	B3 NSGA-II	60	27.63	28.34
Nobel-Germany	B5 greedy	60	27.29	0.09

S9. Without the Wear Price

With the default prices, a sleeping link earns a wear credit about as large as its energy saving (Section 5), so J is well below the energy of the network. To check that this assumed credit does not drive the comparison, we repeat the main study with $\lambda_W = 0$: oracle labels, tuning of B6 and

Table S6: Scalability of the Optimization Step and the Safety Check

Topology	$\|V\|$	$\|E\|$	OD pairs	ILP (s)	Greedy (s)	Safety check (ms)
Abilene	12	15	132	0.48	0.04	0.067
Nobel-Germany	17	26	272	0.41	0.19	0.130
GÉANT	22	36	462	4.07	0.56	0.157
Germany50	50	88	2450	1.61	1.45	0.223
TA2	65	108	4160	11.37	7.42	0.363

B7, training of B4 and LiRA, and the reported cost then contain energy and reconfiguration only. Table S7 shows the result for all 12 settings. The oracle holds a single configuration in 9 of them. On Nobel-Germany B4 is again the most expensive online method (1.1% above the oracle in the base scenario), while B6, B2, the static rule, and LiRA lie within 0.12% of the oracle in every setting. LiRA is cheaper than every other online method in 0 settings and costs the same as the static rule in 10. Under high load it is slightly more expensive than the static rule (1570.2 against 1569.6, which equals the oracle). The ranking of the methods therefore does not depend on the wear model.

Table S7: Repetition of the Main Study Without a Wear Price ($\lambda_W = 0$, All Methods Retrained): Configuration Changes per Day and Cost J (Energy plus Reconfiguration). B4, B6, and LiRA: Mean over Five Seeds

		Changes/d				Cost J with $\lambda_W = 0$					
Topology	Scenario	B4	B6	LiRA	O*	B4	B6	B2	Static	LiRA	O*
Abilene	base	0.02	0.00	0.00	0.00	9151.3	9151.0	9154.7	9151.0	9151.0	9151.0
Abilene	high load	0.00	0.00	0.00	0.00	9157.3	9157.3	9161.0	9157.3	9157.3	9157.3
Abilene	high dynamic power	0.02	0.00	0.00	0.00	9172.3	9172.0	9183.1	9172.0	9172.0	9172.0
Abilene	stress	0.00	0.00	0.00	0.00	9190.9	9190.9	9201.5	9190.9	9190.9	9190.9
GÉANT	base	0.00	0.00	0.00	0.00	1897.8	1897.8	1897.8	1897.8	1897.8	1897.8
GÉANT	high load	0.00	0.00	0.00	0.00	1904.4	1904.4	1904.4	1904.4	1904.4	1904.4
GÉANT	high dynamic power	0.00	0.00	0.00	0.00	1919.1	1919.1	1919.1	1919.1	1919.1	1919.1
GÉANT	stress	0.00	0.00	0.00	0.00	1938.9	1938.9	1938.9	1938.9	1938.9	1938.9
Nobel-Germany	base	11.68	0.13	0.27	0.00	1582.5	1566.0	1566.0	1566.1	1566.1	1565.9
Nobel-Germany	high load	1.76	1.07	0.88	0.80	1572.4	1571.1	1570.0	1569.6	1570.2	1569.6
Nobel-Germany	high dynamic power	11.68	0.27	0.27	0.27	1590.3	1573.9	1574.0	1573.9	1573.9	1573.9
Nobel-Germany	stress	1.76	1.07	0.93	0.80	1584.8	1583.5	1583.1	1584.1	1583.7	1583.0

S10. Traffic Drift

We inject three shifts into the test period of each topology: a ramp that raises all demands by up to 50% over ten days, a fourfold increase of three large demands, and a six-hour shift of the daily pattern. Table S8 compares a LiRA policy trained once with drift-triggered retraining. On Abilene and GÉANT the shifts did not change which configuration this policy uses, because the library stays feasible. On Nobel-Germany the hotspot raised its configuration changes from 0.41 to 0.82 per day. The cost changed by at most 0.74% relative to the unshifted stream. The detectors were also not informative: without any shift, they raised 0.17, 0.00, and 0.93 false alarms per 1000 intervals on Abilene, GÉANT, and Nobel-Germany, and the Page–Hinkley threshold calibrated on validation data was at the maximum of the search grid (200) on the two real traces. We therefore cannot attribute detections to the injected shifts, and retraining did not lower the cost of LiRA. The injected shifts did not change the optimal configuration, so this experiment does not validate the detectors.

Table S8: Injected Traffic Shifts: Post-Shift Cost J of LiRA Trained Once ("Once") and with Drift-Triggered Retraining ("Retrained"). FA/1k Is the Number of False Alarms per 1000 Intervals on the Stream Without a Shift

Topology	Shift	J once	J retrained	Detections	FA/1k
Abilene	growth	4701.5	4701.5	2	0.17
Abilene	hotspot	4698.9	4698.9	2	0.17
Abilene	phase	4698.2	4698.2	2	0.17
GÉANT	growth	394.8	394.8	1	0.00
GÉANT	hotspot	392.6	392.6	0	0.00
GÉANT	phase	392.3	392.3	0	0.00
Nobel-Germany	growth	569.9	570.6	2	0.93
Nobel-Germany	hotspot	573.9	574.5	2	0.93
Nobel-Germany	phase	569.6	570.3	2	0.93

S11. Rolling-Origin Folds

Table S9 repeats the comparison for four rolling origins, retraining B4, B6, and LiRA on all data before each origin. In every fold B4 costs more than B6 and LiRA, and B6 and LiRA stay within 0.35% of the oracle. On Abilene

and GÉANT, all methods coincide in every fold. On Nobel-Germany, the mean cost over the folds is 371.5 for B4, 350.1 for B6, 357.9 for B7, 350.3 for LiRA, and 350.0 for the oracle.

Table S9: Rolling-Origin Folds (Mean over Four Folds, Default Price)

Topology	Method	S (%)	Chg/d	J (mean±std over folds)
Abilene	B4	23.98	0.00	2890.5±0.9
	B6	23.98	0.00	2890.5±0.9
	LiRA	23.98	0.00	2890.5±0.9
	O*	23.98	0.00	2890.5±0.9
GÉANT	B4	29.84	0.00	241.7±0.7
	B6	29.84	0.00	241.7±0.7
	LiRA	29.84	0.00	241.7±0.7
	O*	29.84	0.00	241.7±0.7
Nobel-Germany	B4	27.64	11.17	371.5±5.4
	B6	27.64	0.25	350.1±0.5
	LiRA	27.64	0.33	350.3±0.7
	O*	27.63	0.17	350.0±0.5

S12. Emulation of Configuration Changes

We emulate the most active day of the Nobel-Germany series in Mininet 2.3.0 with Open vSwitch 3.7.1 (17 switches, one host per switch, 26 links). Each configuration installs, for every origin–destination pair, the path that carries the largest share of its demand as OpenFlow rules with `ovs-ofctl`. A day is replayed with two seconds per interval. A configuration change wakes the links that the new configuration needs, waits one second of modeled wake-up time (Mininet does not model line-card physics), replaces the flow tables of the switches that change one after another, and then puts unused links to sleep. Eight UDP probe flows at 100 packets per second on the largest origin–destination pairs measure loss and one-way delay. B4 makes 18 configuration changes on this day, each toggling two links, whereas the sequences of B6, LiRA, and the oracle make none. Each change of B4 takes 1.11 s in total, which is the modeled wake-up time of 1 s plus 99 ms of measured rule installation and the time to put links to sleep. Table S10 reports the probe loss. Each run sends 460 800 probe packets. B4 loses 3.7 packets per configuration change on average, all of them inside the windows around

a change: 0.22% of the probes in those windows and 0.014% over the whole day. The always-on control and LiRA lose no probe packets, and LiRA keeps the same link-hours as B4, computed from the replayed configurations, which are 31% fewer than those of the always-on network. This emulation measures the cost of a single configuration change, not the energy of the network, and it ignores line-card boot time beyond the modeled delay.

Table S10: Mininet Emulation of the Most Active Day of Nobel-Germany (B4: Three Runs; B0 and LiRA: One Run). Trans.: Configuration Changes. Loss in Windows Counts Probes Sent from 0.2 s Before a Change Starts to 1 s After It Ends. Link-Hours Are Computed from the Replayed Configurations

Method	Runs	Trans.	Rule install (ms)	Trans. time (s)	Loss (%)	Loss in windows (%)	Delay p50 (ms)	p99 (ms)	Link-hours
B0	1	0	–	–	0.000	–	0.10	0.4	4.16
B4	3	18	99	1.11	0.014	0.22	0.10	0.3	2.88
LiRA	1	0	–	–	0.000	–	0.09	0.2	2.88

S13. Cost of Ignoring the Sequence

For two configurations c_a and c_b, let $s_{ab} = \lambda_R R(c_a, c_b) + \lambda_W \chi \|x_a - x_b\|_1$ be the non-energy cost of switching between them.

Proposition 1. *Let $c_a, c_b \in \mathcal{C}$ be feasible in every interval and have the same number of sleeping links, and let the traffic alternate so that $E(c_a, D_t) - E(c_b, D_t) = (-1)^t \epsilon$ with $\epsilon > 0$, while every other library member is dominated in every interval. Then the snapshot rule (7) incurs a regret of at least $(T-1)s_{ab} - \lceil T/2 \rceil \epsilon$ with respect to the optimum of (6), which grows linearly in T whenever $s_{ab} > \epsilon/2$.*

Proof. The snapshot rule selects c_b in even and c_a in odd intervals, so it pays the lower energy in each interval and switches $T-1$ times. The sleep terms of (5) are identical for c_a and c_b, so the two sequences differ only in energy and in switching cost. Staying at c_a throughout pays at most $\lceil T/2 \rceil \epsilon$ more energy and never switches. Its cost is an upper bound on the optimum, which proves the claim. □

The proposition shows that flapping can be costly in principle. Whether it is costly in practice depends on whether real traffic produces such near-ties, which is the empirical question of Section 6.

S14. Implementation and Parameters

The flow-level simulator is written in Python with NumPy and sparse path–link matrices. Integer and linear programs are solved with HiGHS (highspy 1.15.1). Experiments ran on an Intel Core i9-13900H laptop (14 cores, 20 threads, 48 GB RAM, no GPU) under Windows, and the emulation ran in Ubuntu on WSL2 with Mininet 2.3.0 and Open vSwitch 3.7.1. Table S11 lists the parameters. The port power values are representative assumptions within the ranges reported in [9, 7], not measurements of a specific device, and the wear parameters are assumed values for the model of [35]. We therefore study the sensitivity to both (Section S7).

Table S11: Model and Training Parameters

Parameter	Value
Threshold θ; paths k; library K	0.8; 4; 32 (default)
Port power $P^{\mathrm{idle}}/P^{\mathrm{dyn}}/P^{\mathrm{sl}}$	30/5/3 W
Wear α; χ	0.5; 0.01 per transition per day
Prices λ_R; λ_W	$0.002\,n_d$; $0.0008\,n_d/\chi$
Interval Δ; n_d	5 min, 288 (GÉANT: 15 min, 96)
XGBoost	depth 6, rate 0.1, 150 rounds max, early stop 15
Features	PCA 95% variance, lags 1/3/6
DAgger rounds; β_1; weight clip	3; 0.5; $[0.2, 5]$
ADWIN confidence; Page–Hinkley tolerance, forgetting	0.002; 0.005, 0.9999
Retraining window; schedule	7 days; at detection, +1 and +3 days
Seeds; folds	5; 4 rolling origins

S15. Noise Structure of the Semi-Synthetic Series

The only series with flapping is semi-synthetic, and its noise is independent across intervals. To see how the findings depend on this choice, we generate three further 30-day series with the same gravity model, weekly

profile, split, and scaling. Two use log-normal noise with the same marginal scale but AR(1) correlation in time ($\rho = 0.5$ and $\rho = 0.9$), and one uses independent noise with half the scale. Each variant gets its own library, and every method is retrained (base scenario, five seeds). Table S12 shows that the amount of flapping depends strongly on the noise. With $\rho = 0.5$ the snapshot classifier changes configuration 57.8 times per day and costs 32.5% more than the oracle, because demands now stay near a decision boundary for several intervals and cross it repeatedly. With $\rho = 0.9$ the oracle itself switches (0.93 changes per day) and B4 is only 0.6% above it, and with half the noise B4 does not flap at all. In every variant, B6, B2, the static rule, and LiRA are at most 0.36% above the oracle, and LiRA costs the same as the static rule. The value of reconfiguration awareness thus ranges from nothing to a third of the cost, depending on the traffic, and in none of the variants does it require learning.

Table S12: Nobel-Germany Series with Different Noise (Base Scenario): Configuration Changes per Day, Total Cost J, and the Excess Cost of B4 over the Oracle. B4, B6, and LiRA: Mean over Five Seeds

	Changes/d				Total cost J						B4 above
Noise	B4	B6	LiRA	O*	B4	B6	B2	Static	LiRA	O*	O* (%)
i.i.d., $\sigma = 1.23$ (main)	11.68	0.13	0.27	0.00	931.7	875.3	875.3	875.9	875.9	874.7	6.5
AR(1) $\rho = 0.5$, $\sigma = 1.23$	57.81	0.00	0.00	0.00	1158.5	874.4	874.4	874.4	874.4	874.4	32.5
AR(1) $\rho = 0.9$, $\sigma = 1.23$	1.92	1.47	1.47	0.93	890.4	888.0	887.7	888.1	888.1	884.9	0.6
i.i.d., $\sigma = 0.62$	0.00	0.00	0.00	0.00	884.2	884.2	884.3	884.2	884.2	884.2	0.0

S16. Decisions on the Previous Interval's Traffic

All results so far let the controller see D_t when it configures interval t (Section 5.3). Table S13 repeats the Nobel-Germany comparison when every online method decides with D_{t-1}: the features, the safety check, the hysteresis and dwell decisions, and the forecast of B2 all use the last completed interval, while costs and violations are measured on D_t (the same trained models, seeds 0 to 2). The decisions barely change, and costs move by at most 0.002% on Nobel-Germany and by less than 0.001% on Abilene and GÉANT. The utilization guarantee does not survive. On Nobel-Germany, B2, B4, and B6 violate θ in 0.05%, 0.19%, and 0.19% of the intervals in the base, high-load, and stress scenarios, the same shares as the always-on network with shortest-path routing, and LiRA in 0.05%, 0.14%, and 0.17%. Every violated interval

of the static rule is one in which the always-on network also violates θ, and in each of them the utilization of the installed configuration rises by 0.66 to 1.36 within one interval. Such peaks come from the independent per-interval noise of the series and cannot be predicted from D_{t-1}. A margin of 0.05 on θ therefore does not remove them, and it raises the cost (LiRA under high load: 886.2 against 880.9). On Abilene and GÉANT at most 0.008% of the intervals violate θ. LiRA's exact match with the oracle under high load thus depends on knowing the traffic of the interval it configures.

Table S13: Nobel-Germany: Total Cost J and Share of Intervals Violating θ When Every Online Method Knows D_t or Decides with D_{t-1}, Without and With a Safety Margin of 0.05 (B4, B6, and LiRA: Mean over Seeds 0 to 2)

	D_t known	D_{t-1}		D_{t-1}, margin 0.05	
Method	J	J	Viol. (%)	J	Viol. (%)
Nobel-Germany, base					
B2 rolling horizon	875.3	875.3	0.05	–	–
B4 snapshot ML	933.7	933.7	0.05	933.7	0.05
B6 hysteresis	875.3	875.3	0.05	875.3	0.05
Static + fallback	875.9	875.8	0.05	875.8	0.05
LiRA	875.9	875.8	0.05	875.8	0.05
Oracle O*	874.7	–	–	–	–
Nobel-Germany, high load					
B2 rolling horizon	882.2	882.2	0.19	–	–
B4 snapshot ML	891.2	891.2	0.19	897.1	0.19
B6 hysteresis	887.0	887.0	0.19	890.7	0.19
Static + fallback	880.9	880.9	0.14	886.2	0.14
LiRA	880.9	880.9	0.14	886.2	0.14
Oracle O*	880.9	–	–	–	–
Nobel-Germany, stress					
B2 rolling horizon	895.4	895.3	0.19	–	–
B4 snapshot ML	903.7	903.7	0.19	909.6	0.19
B6 hysteresis	899.4	899.4	0.19	903.9	0.19
Static + fallback	902.0	902.0	0.19	909.6	0.19
LiRA	897.4	897.4	0.17	901.4	0.17
Oracle O*	894.3	–	–	–	–

S17. Notation, Parameters, and Additional Figures

References

[1] A. P. Bianzino, C. Chaudet, D. Rossi, J.-L. Rougier, A survey of green networking research, IEEE Communications Surveys & Tutorials 14 (1) (2012) 3–20. doi:10.1109/SURV.2011.113010.00106.

Table S14: Main Notation

Symbol	Meaning
$G = (V, E)$	Network graph; E is the set of undirected links
κ_e	Capacity of link e in each direction
Δ, n_d	Control interval; number of intervals per day
D_t, $\hat{D}_t$	Traffic matrix of interval t; its estimate
$c = (x, r)$	Configuration: link states $x \in \{0,1\}^{\lvert E \rvert}$, routing r
$\mathcal{C}$, K	Configuration library and its size
u_e, θ	Utilization of link e; utilization threshold
$E(c, D)$	Energy of c under traffic D in one interval
$R(c', c)$	Reconfiguration cost of moving from c' to c
AF_e, α, χ	Acceleration factor; sleep factor; cycling weight
$W(c', c)$	Per-step wear cost
λ_R, λ_W	Prices of reconfiguration and wear
J, V_t, Q_t	Total cost; cost-to-go; action value

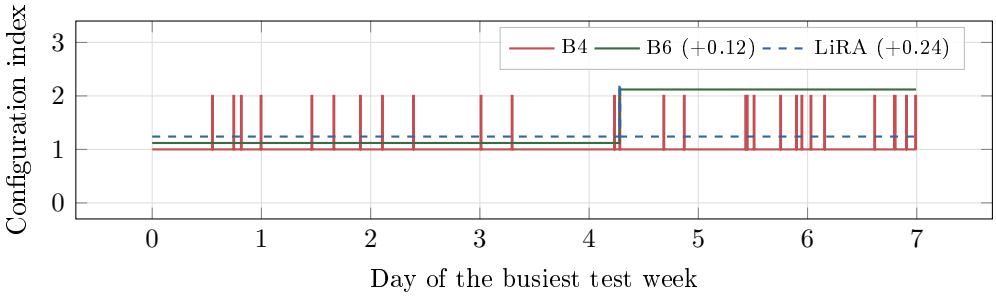


Figure S2: Installed configuration index over the busiest test week of the Nobel-Germany series. Offsets separate the lines. B4 switches repeatedly between two configurations, whereas B6 and LiRA hold a configuration.

[2] R. Bolla, R. Bruschi, F. Davoli, F. Cucchietti, Energy efficiency in the future internet: A survey of existing approaches and trends in energy-aware fixed network infrastructures, IEEE Communications Surveys & Tutorials 13 (2) (2011) 223–244. doi:10.1109/SURV.2011.071410.00073.

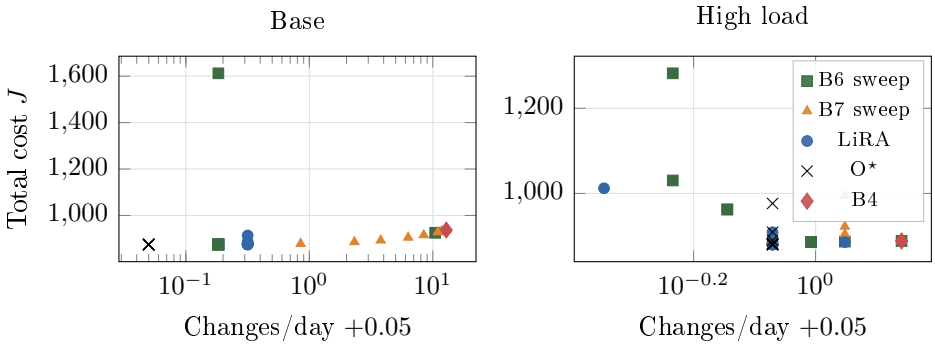


Figure S3: Total cost versus configuration changes on Nobel-Germany. Hysteresis thresholds and dwell times are swept for B6 and B7 and the reconfiguration price is swept for LiRA and the oracle. Each marker is one setting on the test split. The horizontal axis is logarithmic, and 0.05 is added to the change rate so that methods without changes can be shown.

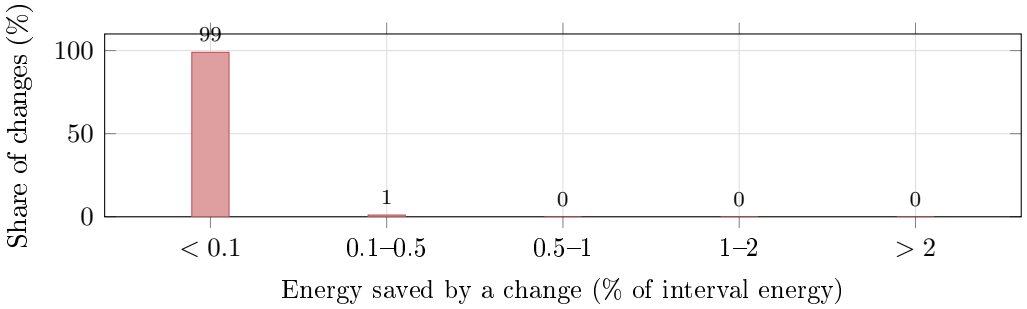


Figure S4: Energy gained by each configuration change of the snapshot classifier B4 (seed 0) on the Nobel-Germany test period (96 changes). 99% of the changes save less than 0.1% of the interval energy.

Table S15: Topologies and Traffic Data

Topology	$\lvert V\rvert$	$\lvert E\rvert$	Δ	Intervals	Span	Traffic
Abilene	12	15	5 min	48 108	2004-03-01 to 2004-09-10	real (SNDlib)
GÉANT	22	36	15 min	10 784	2005-05-04 to 2005-08-31	real (SNDlib)
Nobel-Germany	17	26	5 min	8 640	2004-03-01 to 2004-03-30	semi-synthetic

[3] L. Chiaraviglio, M. Mellia, F. Neri, Minimizing ISP network energy cost: Formulation and solutions, IEEE/ACM Transactions on Networking 20 (2) (2012) 463–476. doi:10.1109/TNET.2011.2161487.

[4] M. Zhang, C. Yi, B. Liu, B. Zhang, GreenTE: Power-aware traffic engineering, in: Proc. IEEE Int. Conf. Network Protocols (ICNP), 2010, pp. 21–30. doi:10.1109/ICNP.2010.5762751.

[5] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, N. McKeown, ElasticTree: Saving energy in data center networks, in: Proc. USENIX NSDI, 2010.
URL https://www.usenix.org/conference/nsdi10-0/elastictree-saving-energy-data-center-networks

[6] N. Vasić, D. Kostić, Energy-aware traffic engineering, in: Proc. ACM e-Energy, 2010. doi:10.1145/1791314.1791341.

[7] P. Mahadevan, P. Sharma, S. Banerjee, P. Ranganathan, A power benchmarking framework for network devices, in: Proc. IFIP Networking, Vol. 5550 of LNCS, 2009, pp. 795–808. doi:10.1007/978-3-642-01399-7_62.

[8] J. C. C. Restrepo, C. G. Gruber, C. M. Machuca, Energy profile aware routing, in: Proc. IEEE ICC Workshops, 2009. doi:10.1109/ICCW.2009.5208041.

[9] A. Vishwanath, K. Hinton, R. W. A. Ayre, R. S. Tucker, Modeling energy consumption in high-capacity routers and switches, IEEE Journal on Selected Areas in Communications 32 (8) (2014) 1524–1532. doi:10.1109/JSAC.2014.2335312.

[10] R. Maaloul, R. Taktak, L. Chaari, B. Cousin, Energy-aware routing in carrier-grade Ethernet using SDN approach, IEEE Transactions on Green Communications and Networking 2 (3) (2018) 844–858. doi:10.1109/TGCN.2018.2832658.

[11] B. G. Assefa, Ö. Özkasap, A survey of energy efficiency in SDN: Software-based methods and optimization models, Journal of Network and Computer Applications 137 (2019) 127–143. doi:10.1016/j.jnca.2019.04.001.

[12] B. G. Assefa, Ö. Özkasap, MER-SDN: Machine learning framework for traffic aware energy efficient routing in SDN, in: Proc. IEEE DASC/PiCom/DataCom/CyberSciTech, 2018. doi:10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.000-1.

[13] P. Fröhlich, E. Gelenbe, M. P. Nowak, Reinforcement learning and energy-aware routing, in: Proc. ACM FlexNets Workshop, 2021. doi:10.1145/3472735.3473390.

[14] X. Zheng, W. Huang, S. Wang, J. Zhang, H. Zhang, Research on energy-saving routing technology based on deep reinforcement learning, Electronics 11 (13) (2022) 2035. doi:10.3390/electronics11132035.

[15] Q. Wu, X. Chen, Z. Zhou, L. Chen, J. Zhang, Deep reinforcement learning with spatio-temporal traffic forecasting for data-driven base station sleep control, IEEE/ACM Transactions on Networking 29 (2) (2021) 935–948. doi:10.1109/TNET.2021.3053771.

[16] Y. Wang, Y. Li, T. Wang, G. Liu, Towards an energy-efficient data center network based on deep reinforcement learning, Computer Networks 210 (2022) 108939. doi:10.1016/j.comnet.2022.108939.

[17] X. Pei, P. Sun, Y. Hu, D. Li, B. Chen, L. Tian, Enabling efficient routing for traffic engineering in SDN with deep reinforcement learning, Computer Networks 241 (2024) 110220. doi:10.1016/j.comnet.2024.110220.

[18] I. Maity, S. Misra, C. Mandal, ETHoS: Energy-aware traffic engineering for sustainable hybrid SDN, IEEE Transactions on Sustainable Computing 7 (4) (2022) 875–886. doi:10.1109/TSUSC.2022.3164571.

[19] L. Röllin, R. Jacob, L. Vanbever, A sleep study for ISP networks: Evaluating link sleeping on real world data, ACM SIGEnergy Energy Informatics Review 4 (2024) 51–58. doi:10.1145/3727200.3727209.

[20] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, On optimizing energy efficiency in SDN networks through ML-driven configuration, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2025. doi:10.1109/NOMS57970.2025.11073657.

[21] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, J. Galán-Jiménez, AI-driven scalable framework for energy-efficient SDN configurations, IEEE Transactions on Green Communications and Networking (2026). doi:10.1109/TGCN.2026.3724271.

[22] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. doi:10.1016/j.ejor.2020.07.063.

[23] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, M. Yu, Teal: Learning-accelerated optimization of WAN traffic engineering, in: Proc. ACM SIGCOMM, 2023. doi:10.1145/3603269.3604857.

[24] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, A. Tamar, DOTE: Rethinking (predictive) WAN traffic engineering, in: Proc. USENIX NSDI, 2023.
URL https://www.usenix.org/conference/nsdi23/presentation/perry

[25] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, P. Bailis, Contracting wide-area network topologies to solve flow problems quickly, in: Proc. USENIX NSDI, 2021.
URL https://www.usenix.org/conference/nsdi21/presentation/abuzaid

[26] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, M. Zaharia, Solving large-scale granular resource allocation problems efficiently with POP, in: Proc. ACM SOSP, 2021, pp. 521–537. doi:10.1145/3477132.3483588.

[27] A. Valadarsky, M. Schapira, D. Shahaf, A. Tamar, Learning to route, in: Proc. ACM HotNets, 2017, pp. 185–191. doi:10.1145/3152434.3152441.

[28] P. Almasan, S. Xiao, X. Cheng, X. Shi, P. Barlet-Ros, A. Cabellos-Aparicio, ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning, Computer Networks 214 (2022) 109166. doi:10.1016/j.comnet.2022.109166.

[29] G. Bernárdez, J. Suárez-Varela, A. López, X. Shi, S. Xiao, X. Cheng, P. Barlet-Ros, A. Cabellos-Aparicio, MAGNNETO: A graph neural network-based multi-agent system for traffic engineering, IEEE Transactions on Cognitive Communications and Networking 9 (2) (2023) 494–506. doi:10.1109/TCCN.2023.3235719.

[30] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, A. Cabellos-Aparicio, RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN, IEEE Journal on Selected Areas in Communications 38 (10) (2020) 2260–2270. doi:10.1109/JSAC.2020.3000405.

[31] M. Ye, J. Zhang, Z. Guo, H. J. Chao, FlexDATE: Flexible and disturbance-aware traffic engineering with reinforcement learning in software-defined networks, IEEE/ACM Transactions on Networking 31 (4) (2023) 1433–1448. doi:10.1109/TNET.2022.3217083.

[32] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, D. Walker, Abstractions for network update, in: Proc. ACM SIGCOMM, 2012, pp. 323–334. doi:10.1145/2342356.2342427.

[33] X. Jin, H. H. Liu, R. Gandhi, S. Kandula, R. Mahajan, M. Zhang, J. Rexford, R. Wattenhofer, Dynamic scheduling of network updates, in: Proc. ACM SIGCOMM, 2014, pp. 539–550. doi:10.1145/2619239.2626307.

[34] E. Bonetto, L. Chiaraviglio, F. Idzikowski, E. Le Rouzic, Algorithms for the multi-period power-aware logical topology design with reconfiguration costs, Journal of Optical Communications and Networking 5 (5) (2013) 394–410. doi:10.1364/JOCN.5.000394.

[35] L. Chiaraviglio, P. Wiatr, P. Monti, J. Chen, J. Lorincz, F. Idzikowski, M. Listanti, L. Wosinska, Is green networking beneficial in terms of device lifetime?, IEEE Communications Magazine 53 (5) (2015) 232–240. doi:10.1109/MCOM.2015.7105670.

[36] L. Chiaraviglio, et al., Impact of energy-efficient techniques on a device lifetime, in: Proc. IEEE Online Conference on Green Communications (OnlineGreenComm), 2014. doi:10.1109/OnlineGreenCom.2014.7114426.

[37] P. Wiatr, et al., Lifetime-aware provisioning in green optical backbone networks, in: Proc. Optical Fiber Communication Conference (OFC), 2016. doi:10.1364/OFC.2016.M3K.1.

[38] A. Borodin, N. Linial, M. E. Saks, An optimal on-line algorithm for metrical task system, Journal of the ACM 39 (4) (1992) 745–763. doi:10.1145/146585.146588.

[39] N. Chen, G. Goel, A. Wierman, Smoothed online convex optimization in high dimensions via online balanced descent (2018). arXiv:1803.10366. URL https://arxiv.org/abs/1803.10366

[40] M. Lin, A. Wierman, L. L. H. Andrew, E. Thereska, Dynamic right-sizing for power-proportional data centers, IEEE/ACM Transactions on Networking 21 (5) (2013) 1378–1391. doi:10.1109/TNET.2012.2226216.

[41] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proc. AISTATS, 2011, pp. 627–635.
URL https://proceedings.mlr.press/v15/ross11a.html

[42] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, Journal of the ACM 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.

[43] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, ACM Transactions on Algorithms 19 (2) (2023) 19:1–19:34. doi:10.1145/3582689.

[44] N. Christianson, J. Shen, A. Wierman, Optimal robustness-consistency tradeoffs for learning-augmented metrical task systems, in: Proc. 26th Int. Conf. Artificial Intelligence and Statistics (AISTATS), Vol. 206 of PMLR, 2023, pp. 9377–9399.

[45] D. Rutten, N. Christianson, D. Mukherjee, A. Wierman, Smoothed online optimization with unreliable predictions, Proc. ACM Meas. Anal. Comput. Syst. 7 (1) (2023) 12:1–12:36. doi:10.1145/3579442.

[46] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Computing Surveys 46 (4) (2014) 44:1–44:37. doi:10.1145/2523813.

[47] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, IEEE Transactions on Knowledge and Data

Engineering 31 (12) (2019) 2346–2363. doi:10.1109/TKDE.2018.2876857.

[48] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: Proc. SIAM Int. Conf. Data Mining (SDM), 2007, pp. 443–448. doi:10.1137/1.9781611972771.42.

[49] E. S. Page, Continuous inspection schemes, Biometrika 41 (1/2) (1954) 100–115. doi:10.1093/biomet/41.1-2.100.

[50] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, L. Cavallaro, INSOMNIA: Towards concept-drift robustness in network intrusion detection, in: Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2021, pp. 111–122. doi:10.1145/3474369.3486864.